

ОШИБКИ И ОТЛАДКА В PYTHON.



Ошибки — это нормально

Когда мы только начинаем программировать, ошибки могут пугать. Красные сообщения, непонятные слова, программа «ломается». Но на самом деле ошибки — это абсолютно нормальная часть программирования.

Все программисты — даже самые опытные — делают ошибки. Важно не бояться их, а научиться понимать, находить и исправлять. Это называется **отладкой** (debugging).

Ошибки — это наш друг. Они подсказывают, где и что пошло не так. Главное — научиться их читать.

ОШИБКИ И ОТЛАДКА В PYTHON.



1. Синтаксическая ошибка (SyntaxError)

Это ошибка в написании кода. Python не может понять, что вы хотели сказать.

```
print("Привет"
```

1. Синтаксическая ошибка (SyntaxError)

Это ошибка в написании кода. Python не может понять, что вы хотели сказать.

```
print("Привет"
```

Ошибка: закрывающая скобка отсутствует. Python напишет:

```
File "/home/runner/workspace/main.py", line 1
    print("Привет"
          ^
SyntaxError: '(' was never closed
```

1. Синтаксическая ошибка (SyntaxError)

Пример другой ошибки:

```
if x > 5  
    print("Больше пяти")
```

1. Синтаксическая ошибка (SyntaxError)

Пример другой ошибки:

```
if x > 5  
    print("Больше пяти")
```

Ошибка: пропущен двоеточие : после условия. Python скажет:

```
File "/home/runner/workspace/main.py", line 1  
    if x > 5  
        ^  
SyntaxError: expected ':'
```

2. Ошибка выполнения (RuntimeError)

Код написан правильно, но при запуске что-то идёт не так.

```
x = 10 / 0
```


2. Ошибка выполнения (RuntimeError)

Код написан правильно, но при запуске что-то идёт не так.

```
x = 10 / 0
```

Ошибка: деление на ноль. Python сообщает:

```
File "/home/runner/workspace/main.py", line 1, in <module>
    x = 10 / 0
        ~~~^~
ZeroDivisionError: division by zero
```

3. Ошибка логики

Самая сложная: программа работает, но **делает не то**, что вы хотели.

```
def area(length, width):  
    return length + width # а должно быть умножение!
```

Ошибок нет, программа запускается — но результат неправильный.

Как Python сообщает об ошибках

Вывод ошибок из примеров выше — это **Traceback**.

Traceback — это цепочка, показывающая:

- где именно произошла ошибка;
- какие функции и строки кода были вызваны перед этим;
- типошибки (например, `ZeroDivisionError`, `IndexError`, `SyntaxError` и т. д.);
- короткое описание ошибки (например, `division by zero` — деление на ноль).

Как Python сообщает об ошибках

Пример:

```
numbers = [1, 2, 3]  
print(numbers[5])
```

Ошибка:

```
File "/home/runner/workspace/main.py", line 2, in <module>  
    print(numbers[5])  
            ~~~~~^  
IndexError: list index out of range
```

Как Python сообщает об ошибках

Traceback — один из вариантов вывода ошибок после запуска программы, однако Многие современные редакторы кода, такие как:

- PyCharm
- Thonny (для начинающих)
- Replit
- Jupyter Notebook
- **VS Code**

умеют показывать ошибки ещё до запуска программы. Это называется **статический анализ кода**.

Как Python сообщает об ошибках

1. Подчёркивание переменных, которых не существует

```
1 name = "Имя"
2 print(nmae) # опечатка!
```

В редакторе «nmae» будет подчёркнуто красной волнистой линией: переменная не определена.

```
Traceback (most recent call last):
  File "c:\Users\Home\Downloads\all (17).py", line 2, in <module>
    print(nmae) # опечатка!
      ^^^^^
NameError: name 'nmae' is not defined
```

Как Python сообщает об ошибках

2. Подсказка, что забыли : в if

```
1 if x > 5
2     print("Большое число")
```

Редактор скажет: «SyntaxError — ожидается ':'», и сам подсветит строку.

```
File "c:\Users\Home\.vscode\extensions\ms-python.
code = compile(f.read(), fname, "exec")
File "C:\Users\Home\Downloads\05_1.py", line 1
if x > 5
    ^
SyntaxError: expected ':'
```

Как Python сообщает об ошибках

3. Несовместимые типы

```
1 print("Возраст: " + 15)
```

Редактор скажет: «Нельзя сложить str и int».

```
Traceback (most recent call last):  
  File "c:\Users\Home\Downloads\05_1.py", line 1, in <module>  
    print("Возраст: " + 15)  
                        ^~~~~~  
TypeError: can only concatenate str (not "int") to str
```


Как это работает?

Редакторы анализируют код на предмет:

- опечаток
- пропущенных скобок
- неправильного отступа
- конфликтов типов
- использования несуществующих переменных

Они делают это ещё до запуска, помогая избежать ошибок заранее.

Как читать ошибки: пошагово

```
1 name = input("Введите имя: ")
2 print("Привет, " + names)
```

```
Введите имя: wd
```

```
Traceback (most recent call last):
```

```
File "c:\Users\Home\Downloads\all (17).py", line 2, in <module>
```

```
    print("Привет, " + names)
```

```
        ^^^^^
```

```
NameError: name 'names' is not defined. Did you mean: 'name'?
```

Вывод: внимательно читаем сообщение, смотрим, что за переменная или функция вызывает ошибку.

Общий вид конструкции **try-except**

```
try:
    # опасный код
except ТипОшибки1:
    # обработка ошибки 1
except ТипОшибки2:
    # обработка ошибки 2
else:
    # если ошибок не было
finally:
    # выполняется всегда
```

Использование конструкции **try-except**

Иногда вы ожидаете, что ошибка может возникнуть, и хотите обработать её красиво. Для этого используется конструкция **try-except**:

```
1 try:
2     x = int(input("Введите число: "))
3     print(10 / x)
4 except ValueError:
5     print("Вы ввели не число!")
6 except ZeroDivisionError:
7     print("Нельзя делить на ноль!")
```

- **try**: в этом блоке мы пишем код, который может вызвать ошибку.
- **except ValueError**: этот блок сработает, если произойдёт ошибка преобразования строки в число.
- **except ZeroDivisionError**: сработает, если пользователь введёт 0.

Блок else

Выполняется только если не было ошибок:

```
1 try:
2     x = 5
3 except:
4     print("Ошибка!")
5 else:
6     print("Ошибки не было!")
```

Блок finally

Выполняется в любом случае, даже если была ошибка:

```
1 try:
2     x = 1 / 0
3 except:
4     print("Ошибка!")
5 finally:
6     print("Завершение")
```

Советы по отладке

Отладка — это процесс поиска и исправления ошибок.

1. Читайте ошибку сверху вниз.
2. Ищите строку кода, где ошибка.
3. Проверяйте переменные: названы ли правильно?
4. Ставьте `print()` — смотрите, что происходит.
5. Делайте маленькие шаги.
6. Комментируйте части кода временно, чтобы изолировать ошибку.
7. обращайтесь внимание на подсветку строк от программы, она предостерегает от ошибок.

Типичные ошибки и как их решать

IndexError

```
numbers = [1, 2, 3]  
print(numbers[5]) # нет такого индекса!
```

Решение: проверить длину списка с `len(numbers)`.

ValueError

```
age = int(input("Введите возраст: "))
```

Если ввести «привет», будет ошибка. Оберните в try-except.

Типичные ошибки и как их решать

TypeError

```
print("Возраст: " + 10)
```

Нельзя складывать строку и число. Надо:

```
print("Возраст: " + str(10))
```

Программа с ошибками и отладка

Тут код ошибками, давайте попробуем исправить их!

```
items = input("Введите покупки: ").split(",")
print("У вас", len(items), "покупок.")

if "хлеб" in items:
    print("Вы купили хлеб!")
else:
    print("Хлеб не куплен")

print("Вторая покупка:", items[1])
```

Спасибо за внимание!

